

LAB 12 | المخبر 12

Battery Scheduling with Deep Q-Learning

جدولة بطارية باستخدام التعلم العميق Q

A MATLAB Custom Environment for Time-of-Use Energy Management

بيئة MATLAB مخصصة لإدارة الطاقة وفق التعرفة الزمنية

Environment

Model

Workflow

MATLAB Code

Results

Tasks

Prepared by | إعداد | Dr.-Ing. Aouss Gabash



Learning Outcomes

مخرجات التعلم

1. Implement a MATLAB workflow for ai-based load forecasting.
تنفيذ سير عمل في MATLAB حول التنبؤ بالأحمال القائم على الذكاء الاصطناعي.
2. Interpret numerical results, plots, and engineering implications.
تفسير النتائج العددية والرسوم والدلالات الهندسية.
3. Validate the implementation and discuss limitations.
التحقق من صحة التنفيذ ومناقشة حدوده.



Prerequisites

المتطلبات
السابقة

Time-series analysis, regression, and load characteristics.

تحليل السلاسل الزمنية
والانحدار وخصائص
الأحمال.

AI-Based Load Forecasting | التنبؤ

بالأحمال القائم على الذكاء
الاصطناعي



Laboratory Objectives

- Create a custom battery environment with `rlFunctionEnv`.
- Define observations, actions, rewards, and terminal conditions.
- Implement SOC dynamics, energy cost, degradation, and penalties.
- Train a DQN agent and inspect its learned policy.
- Compare the learned schedule with a no-battery baseline.
- Evaluate safety, reward sensitivity, and generalization.

أهداف المخبر

- إنشاء بيئة بطارية مخصصة باستخدام `rlFunctionEnv`.
- تعريف الملاحظات والأفعال والمكافآت وشروط النهاية.
- تنفيذ ديناميكا SOC وكلفة الطاقة والتآكل والعقوبات.
- تدريب وكيل DQN وتحليل سياسته.
- مقارنة الجدولة المتعلمة مع خط أساس دون بطارية.
- تقييم الأمان وحساسية المكافأة والتعميم.

1. Requirements

- MATLAB
- Reinforcement Learning Toolbox
- Deep Learning Toolbox

1. المتطلبات

يلزم MATLAB مع Reinforcement Learning Toolbox و Deep Learning Toolbox.

2. Environment Definition

Observation

SOC, normalized load, normalized price, normalized hour.

Actions

-1 charge, 0 idle, +1 discharge.

Episode

24 hourly steps.

Reward

Negative energy cost, degradation, and violations.

$$st = [SOC_t, Lt, Pricet, Hour_t]$$

$$at \in -1, 0, +1$$

2. تعريف البيئة

تتكون الحالة من SOC والحمل والسعر والساعة، بينما تمثل الأفعال الشحن والتوقف والتفريغ.

3. Battery and Reward Model

$$SOC_{t+1} = SOC_t + \eta_c P_{c,t} \Delta t / E_{cap} - P_{d,t} \Delta t / (\eta_d E_{cap})$$

$$P_{grid,t} = \max(0, P_{load,t} + P_{batt,t})$$

$$r_t = -C_{energy,t} - C_{deg,t} - \lambda_v Violation_t$$

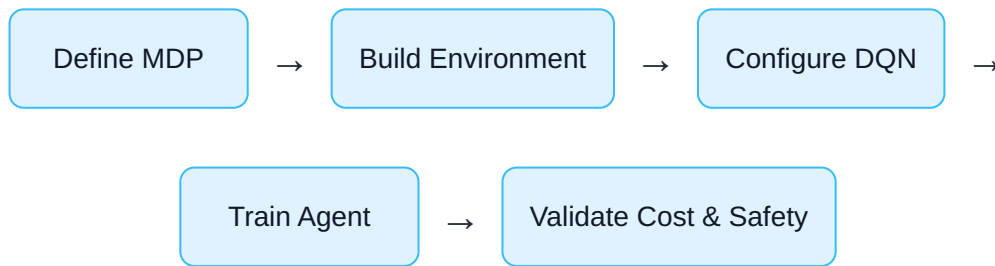
$$r_T \leftarrow r_T - \lambda_f |SOC_T - SOC_{target}|$$

The terminal-SOC penalty prevents the agent from exploiting the episode boundary by emptying the battery at the final hour.

3. نموذج البطارية والمكافأة

تتغير حالة الشحن بحسب كفاءة الشحن والتفريغ، وتُعاقب المكافأة كلفة الطاقة والتآكل وتجاوز الحدود والانحراف النهائي في SOC.

4. Engineering Workflow



1. Specify states, actions, dynamics, and limits.
2. Normalize observations.
3. Define a reward aligned with operating cost and safety.
4. Train with replay memory and target updates.
5. Compare against a no-battery and rule-based baseline.
6. Test changed prices, loads, and battery sizes.

4. سير العمل الهندسي

نعرّف MDP ونبني البيئة ونضبط DQN، ثم ندرب الوكيل ونختبر الكلفة والأمان والتعميم.

5. Main MATLAB Script

```

%% Lab 12: Battery Scheduling using DQN
clear; clc; close all; rng(12);

%% Observation and action specifications
obsInfo = rlNumericSpec([4 1], ...
    LowerLimit=[0;0;0;0], ...
    UpperLimit=[1;1;1;1]);
obsInfo.Name = "battery_observation";

actInfo = rlFiniteSetSpec([-1,0,1]);
actInfo.Name = "battery_action";

%% Create and validate environment
env = rlFunctionEnv(obsInfo,actInfo, ...
    @batteryStepFunction,@batteryResetFunction);
validateEnvironment(env);

%% DQN agent
agent = rlDQNAgent(obsInfo,actInfo);
agent.AgentOptions.UseDoubleDQN = true;
agent.AgentOptions.DiscountFactor = 0.99;
agent.AgentOptions.MinibatchSize = 128;
agent.AgentOptions.ExperienceBufferLength = 1e5;
agent.AgentOptions.TargetUpdateFrequency = 4;
agent.AgentOptions.EpsilonGreedyExploration.Epsilon = 1.0;
agent.AgentOptions.EpsilonGreedyExploration.EpsilonMin = 0.05;
agent.AgentOptions.EpsilonGreedyExploration.EpsilonDecay = 1e-4;

%% Training
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=1200, ...
    MaxStepsPerEpisode=24, ...
    ScoreAveragingWindowLength=50, ...
    StopTrainingCriteria="AverageReward", ...
    StopTrainingValue=-8, ...
    Verbose=false, ...
    Plots="training-progress");
trainingStats = train(agent,env,trainOpts);

%% Simulation
simOpts = rlSimulationOptions(MaxSteps=24);
experience = sim(env,agent,simOpts);
obs = experience.Observation.battery_observation.Data;
act = experience.Action.battery_action.Data;
rew = experience.Reward.Data;

```

```

soc = squeeze(obs(1,1,:));
actions = squeeze(act);
rewards = squeeze(rew);
hours = 0:numel(actions)-1;

figure; stairs(0:numel(soc)-1,soc,'LineWidth',1.4); grid on;
xlabel('Hour'); ylabel('SOC'); title('Battery State of Charge');

figure; stairs(hours,actions,'LineWidth',1.4); grid on;
yticks([-1 0 1]); yticklabels({'Charge','Idle','Discharge'});
xlabel('Hour'); ylabel('Action'); title('DQN Battery Actions');

fprintf('Episode reward = %.3f\n',sum(rewards));

%% Baseline
[~,baseInfo] = batteryResetFunction();
baselineCost = sum(baseInfo.Load.*baseInfo.Price);
fprintf('No-battery daily cost = %.2f EUR\n',baselineCost);

```

5. البرنامج الرئيسي

ينشئ مواصفات البيئة والوكيل، ثم يدرب DQN ويشغله ليوم كامل ويقارن الأداء مع خط الأساس.

6. Reset Function

```

function [InitialObservation,Info] = batteryResetFunction()
Info.Load = [2.2 2.0 1.9 1.8 1.9 2.3 3.0 3.8 ...
             4.2 4.0 3.7 3.5 3.4 3.6 4.0 4.8 ...
             5.8 6.5 6.1 5.4 4.7 4.0 3.2 2.6]';
Info.Price = [0.18*ones(6,1); 0.25*ones(10,1); ...
             0.42*ones(5,1); 0.23*ones(3,1)];
Info.Capacity = 10; Info.Pmax = 2.5;
Info.EtaCharge = 0.95; Info.EtaDischarge = 0.95;
Info.SOCmin = 0.10; Info.SOCmax = 0.90;
Info.Hour = 1; Info.SOC = 0.50; Info.TotalCost = 0;
InitialObservation = makeObservation(Info);
end

```

6. دالة إعادة الضبط

تهينى الحمل والتعرفة ومعاملات البطارية والساعة وحالة الشحن في بداية كل حلقة.

7. Step Function

```

function [NextObservation,Reward,IsDone,Info] = ...
    batteryStepFunction(Action,Info)

action = double(Action); h = Info.Hour;
loadPower = Info.Load(h); price = Info.Price(h);
PbattCommand = action*Info.Pmax;
socOld = Info.SOC; violation = 0;

if PbattCommand < 0
    socNew = socOld + Info.EtaCharge*(-PbattCommand)/Info.Capacity;
elseif PbattCommand > 0
    socNew = socOld - PbattCommand/(Info.EtaDischarge*Info.Capacity);
else
    socNew = socOld;
end

if socNew > Info.SOCmax
    violation = socNew-Info.SOCmax; socNew = Info.SOCmax;
elseif socNew < Info.SOCmin
    violation = Info.SOCmin-socNew; socNew = Info.SOCmin;
end

deltaSOC = socNew-socOld;
if deltaSOC >= 0
    gridBatteryPower = deltaSOC*Info.Capacity/Info.EtaCharge;
else
    gridBatteryPower = deltaSOC*Info.Capacity*Info.EtaDischarge;
end

gridPower = max(0,loadPower+gridBatteryPower);
energyCost = price*gridPower;
degradationCost = 0.015*abs(gridBatteryPower);
Reward = -energyCost-degradationCost-30*violation;

Info.SOC = socNew;
Info.TotalCost = Info.TotalCost+energyCost;
Info.Hour = Info.Hour+1;
IsDone = Info.Hour > 24;

if IsDone
    Reward = Reward-3*abs(Info.SOC-0.50);
    Info.Hour = 24;
end

```

```
end
NextObservation = makeObservation(Info);
end
```

7. دالة الخطوة

تحدّث SOC وتفرض الحدود وتحسب قدرة الشبكة والكلفة والتآكل والمكافأة.

8. Observation Helper

```
function observation = makeObservation(Info)
h = min(Info.Hour,24);
observation = [
    Info.SOC
    Info.Load(h)/max(Info.Load)
    Info.Price(h)/max(Info.Price)
    (h-1)/23
];
end
```

8. بناء الملاحظة

تُطبع قيم الحمل والسعر والساعة وتُجمع مع SOC في متجه واحد.

9. Expected Results and Validation

Low-Price Charging

The battery should charge during inexpensive periods.

Peak Discharging

It should discharge during high-price evening hours.

SOC Safety

No sustained violation of SOC bounds.

Economic Benefit

Total daily cost should beat the no-battery baseline.

- Plot episodic and moving-average reward.

- Plot SOC and actions.
- Compute energy cost, degradation cost, and violations separately.
- Repeat simulation over several randomized days.

Training reward alone is not enough; verify physical feasibility and cost reduction explicitly.

9. النتائج المتوقعة والتحقق

يُتوقع الشحن في الفترات الرخيصة والتفريغ في الذروة مع احترام حدود SOC وتحقيق وفر مقارنة بخط الأساس.

10. Student Tasks

1. Change battery capacity to 5 and 20 kWh.
2. Use five discrete power levels.
3. Add PV generation.
4. Add a demand-charge term.
5. Randomize load and price at reset.
6. Compare DQN with a rule-based controller.
7. Test a changed tariff without retraining.

10. مهام الطالب

1. تغيير سعة البطارية.
2. استخدام خمسة مستويات قدرة.
3. إضافة إنتاج PV.
4. إضافة كلفة للطلب الأعظمي.
5. تغيير الحمل والسعر عشوائيًا.
6. المقارنة مع تحكم قائم على قواعد.
7. اختبار تعرفه مختلفة دون إعادة تدريب.



Research Extension

Microgrid extension: add PV, a diesel generator, grid-import limits, load shedding, and uncertain forecasts. Compare DQN with Double DQN, dueling DQN, PPO, and model-predictive control.

وسّع البيئة إلى شبكة مصغرة تضم PV ومولد ديزل وحدود استيراد وفصل أحمال وتنبؤات غير مؤكدة، ثم قارن خوارزميات متعددة.

Review Questions

1. Why is battery scheduling an MDP?
2. Why are observations normalized?
3. What does the replay buffer do?
4. Why is a target network used in DQN?
5. How does reward scaling affect learning?
6. Why is a terminal-SOC penalty necessary?
7. How should the learned policy be validated?

أسئلة المراجعة

1. لماذا تعد جدولة البطارية مسألة MDP؟
2. لماذا نطبع الملاحظات؟
3. ما دور ذاكرة الخبرة؟
4. لماذا تستخدم شبكة هدف في DQN؟
5. كيف يؤثر مقياس المكافأة في التعلم؟
6. لماذا نحتاج عقوبة SOC النهائية؟
7. كيف نتحقق من السياسة المتعلمة؟

References | المراجع

المراجع العامة المستخدمة في هذا المقرر

- [1] A. Gabash, *Flexible Optimal Operations of Energy Supply Networks with Renewable Energy Generation and Battery Storage*. Saarbrücken, Germany: Südwestdeutscher Verlag für Hochschulschriften, 2014.
- [2] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th Global ed. Harlow, U.K.: Pearson Education Limited, 2022.
- [3] A. Gabash, "Energy Market Transition and Climate Change: A Review of TSOs-DSOs C+++ Framework from 1800 to Present," *Energies*, vol. 16, no. 17, Art. no. 6139, 2023, doi: 10.3390/en16176139.
- [4] A. Gabash, *AI Applications in Electrical Power Systems*, Version 1.0, 2026. [Online]. Available: <https://aoussgabash.com>

Recommended citation:

Gabash, A. (2026). Battery Scheduling with Deep Q-Learning. AI Applications in Electrical Power Systems, Laboratory 12, Version 1.0. <https://aoussgabash.com>

المرجع المقترح:

أوس غباش، 2026، مخبر 12، تطبيقات الذكاء الاصطناعي في أنظمة الطاقة الكهربائية، الإصدار 1.0، <https://aoussgabash.com>

© 2026 Dr.-Ing. Aouss Gabash. Educational use with attribution to the author and official website.